

BSDA2001P: Introduction to DL and GenAI Project : Report

Dhruv Bansal

BS in Data Science and Applications
Indian Institute of Technology Madras
24f1001707@ds.study.iitm.ac.in

Abstract

Answering a Multiple Choice Question is a challenging Natural Language Processing (NLP) task that requires a model to understand context, reason over options, and accurately answer the most probable choice. This project addresses the Smart MCQ Solver Kaggle competition, where the objective is to predict the top three most likely correct answers for each question using MAP@3 (Mean Average Precision at 3) evaluation metric.

To solve this task, multiple architectures were implemented and compared. These include a TFIDF+LightGBM Baseline, a custom attention-based BiLSTM, fine-tuned pre-trained transformer models (BERT, RoBERTa, DeBERTa), and a Zero-Shot BART model. Furthermore, a Retrieval-Augmented Generation (RAG) pipeline incorporating semantic retrieval and reranking was integrated to incorporate better contextual knowledge during inference. During experimentation, the highest public leaderboard score achieved was 0.75311 MAP@3. The experimental results demonstrate that pretrained transformer models outperformed models trained from scratch. While Retrieval-based augmentation provides additional context at the cost of increased computational complexity. Overall this project provided practical experience in modern NLP, transformer fine-tuning, retrieval-augmented systems, and experiment tracking.

1. Introduction

Problem Statement:

Recent developments in the AI field improved machines' ability to understand natural language. MCQ answering is an important NLP task which requires a model to understand the question and analyse the options to pick the most appropriate one based on contextual understanding.

In this project we need to make an AI model which solves the multiple choice questions which have 5 candidate options, and the model needs to understand the question and predict the three most likely correct answers in ranked order.

Project Objectives:

The primary objective of this project was to design and implement different models for this task and gaining experience with the modern NLP techniques, and the tools and frameworks like PyTorch [11], PyTorch Lightning [12], Hugging Face Transformers [10], Sentence Transformers [7], FAISS [8], Weights & Biases (W&B) [14].

Report Structure:

Section 3 describes the dataset, exploratory data analyses, preprocessing pipeline, the strategies used to eliminate data leakage. Section 4 explains the tokenization strategies used across the implemented models. Section 5 presents the architecture and implementation details of different models. Section 6 evaluates and compares their performance using the evaluation metrics. Section 7 summarizes the key findings, discusses the challenges, and possible directions for future work, while Section 8 contains all the references, softwares and tools used in this project.

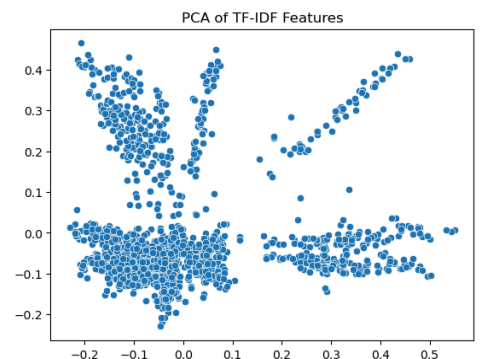
3. Dataset & Preprocessing

Dataset Description: The dataset used in this project was obtained from the Smart MCQ Solver Kaggle Competition [15].

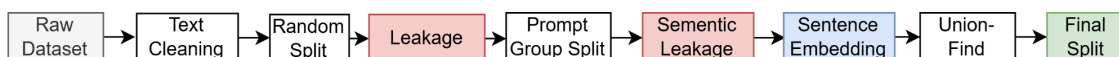
The dataset contains 'id' - unique identifier, 'prompt' - the question statement, 'A-E' - Five possible answer choices, 'answer' - the truth correct option label.

Exploratory Data Analysis (EDA):

- There were no exact duplicates, but 183 duplicate samples after ignoring ordering. And there were 242 prompts which had duplicates.
- Most of the questions were based on physics topics.
- On performing Principal Component Analysis (PCA) on TFIDF vectorized question prompts several clusters were observed suggesting that multiple groups with similar topics may present.
- The maximum word length among the options and the prompt is 118.
- There were 34.9% questions in which the longest option was correct.
- Answer B is the most common correct option, followed by C whereas E is least correct.



Data Preprocessing:



Initially, processed data v001 was created by cleaning the raw text (i.e. removing multi-spaces, \n, \t, \r). After observing data leakage, a StratifiedGroupKFold split on the prompt groups was used in an attempt to eliminate it.

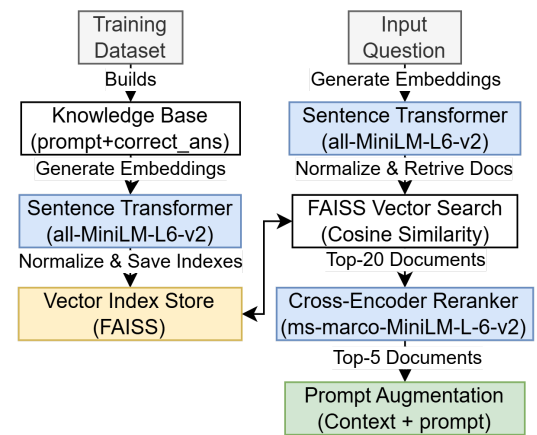
But this method was insufficient to completely eliminate data leakage, as many questions actual text is different but their semantic meaning is same, so a second processed data v002 was made which removed the duplicates (irrespective of option orders), then split the data based on semantic similarity of prompts by encoding the text using the all-MiniLM-L6-v2 [7] Sentence Transformer then grouping similar prompts based on cosine similarity, then splitting using StratifiedGroupKFold.

Data Augmentation:

Initially all the models were trained via making question-prompt pairs, but to provide additional contextual information for the transformer models to perform better, a **Retrieval-Augmented Generation (RAG) Pipeline** was implemented.

The RAG pipeline used here works in two stages: vector index construction and retrieval & prompt augmentation. During the vector index construction, the training dataset after basic cleaning was used to construct a knowledge base by combining each question prompt with its correct answer. Then these documents were converted into vector embeddings via the Sentence Transformer (all-MiniLM-L6-v2) model [7] and indexed using FAISS (Facebook AI Similarity Search) [8].

During the inference stage each input prompt is 1st embedded using the same transformer then similar top-20 semantically similar documents were fetched using the FAISS index search, then those 20 documents were reranked using Cross-Encoder (ms-marco-MiniLM-L-6-v2) [7] and top-5 documents were selected which jointly becomes the context. This context and the input question was combined to make the final input prompt.



4. Tokenization Strategy

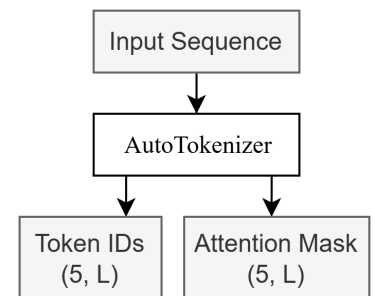
To convert the text to a model-readable format i.e sequence of numerical tokens that can be processed by the models, following strategies were adopted based on the underlying model architecture.

4.1 Transformer-Based Tokenizers

For all the transformer models, the preprocessing pipeline was identical except for the tokenizer itself.

- **Input Sequence:** Each question is split into 5 prompt-option pairs.
- Model specific tokenizer is provided via Hugging Face AutoTokenizer API [10] which tokenizes the input with the input sequence truncated and padded with the max_length.

To ensure compatibility with the pretrained models, the same tokenizer is used, which was used at the time of pre-training which is provided via Hugging Face's AutoTokenizer.from_pretrained() API.



4.1.1 WordPiece Tokenizer for BERT

Pre-trained Model: bert-base-uncased [1]

WordPiece Tokenizer [16] decomposes words into small frequently occurring subword units based on their likelihood in the training corpus. This makes the model to effectively handle unseen or rare words while preserving contextual information making it well suited for natural language understanding tasks.

4.1.2 Byte-Pair Encoding (BPE) for RoBERTa

Pre-trained Model: roberta-base [2]

Byte-Pair Encoding [17] iteratively merges most frequently co-occurring character pairs to construct subword tokens. These flexible subword representations are capable of handling unseen words efficiently.

4.1.3 SentencePiece Unigram for DeBERTa-v3

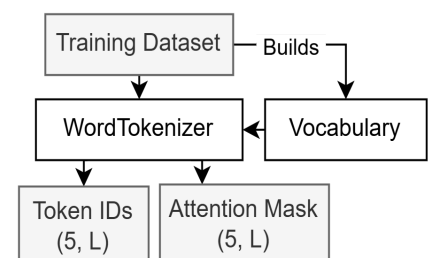
Pre-trained Model: deberta-v3-base [3][4]

SentencePiece [18] tokenizer It starts with a large vocab and iteratively removes the least probable token based on log-likelihood. Performs tokenization independent of whitespaces, making it robust for different text formats.

4.2 Custom Tokenization for the BiLSTM Model

Unlike the transformer models, the BiLSTM architecture requires numerical word indices instead of pretrained subword tokens. Therefore, a custom WordTokenizer is created.

- A Vocabulary class is implemented, which provides the 2-way mapping b/w words & integer indices, it is constructed during pre-processing via iterating through training data.



- A WordTokenizer class is implemented, which utilizes the Vocabulary to convert the text (without punctuations) into a sequence of word indices, and provides an attention mask which tells location of <PAD> tokens.

4.3 Feature Representation for the LightGBM Model

LightGBM is a classical machine learning model which requires numerical feature vectors instead of token IDs, therefore Term Frequency-Inverse Document Frequency (TFIDF) representation was used.

- Used TfidfVectorizer() provided by the Scikit-learn library [13], with configurations - max_features=30000, ngram_range=(1,2), min_df=5, max_df=0.9.
- It assigns higher numerical weights to words which were frequent within a specific document but rare across the entire corpus.

4.4 Experimental Comparison

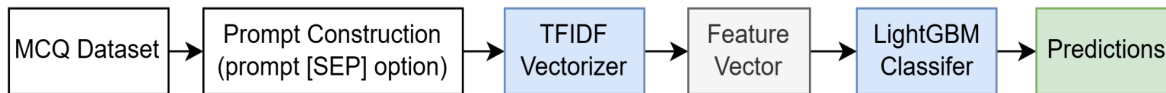
Multiple tokenization strategies were evaluated during this project. The **Byte-Pair Encoding (BPE)** used by RoBERTa achieved the best overall Kaggle performance when paired with its corresponding pretrained model. Therefore, it was selected as the tokenizer for the final model.

5. Modeling & Experimentation

Different modeling techniques were explored for solving this task. The experiments were carried out progressively, beginning with a classical machine learning based LightGBM model, followed by custom BiLSTM based deep learning architecture, and then several pretrained transformer models. In addition to these, a RAG pipeline was implemented to investigate whether providing better context improves the performance.

5.1 Model 1: ML-based TFIDF+LightGBM Model

Architecture:



The sparse feature vectors generated via the TFIDF vectorizer were used to train the LightGBM Classifier (with n_estimators=100, lr=0.1) and then used to make the predictions.

LightGBM [6] is an efficient gradient boosting framework that constructs an ensemble of decision trees using leaf-wise tree growth. It splits the leaf node with the max delta loss.

Salient Features:

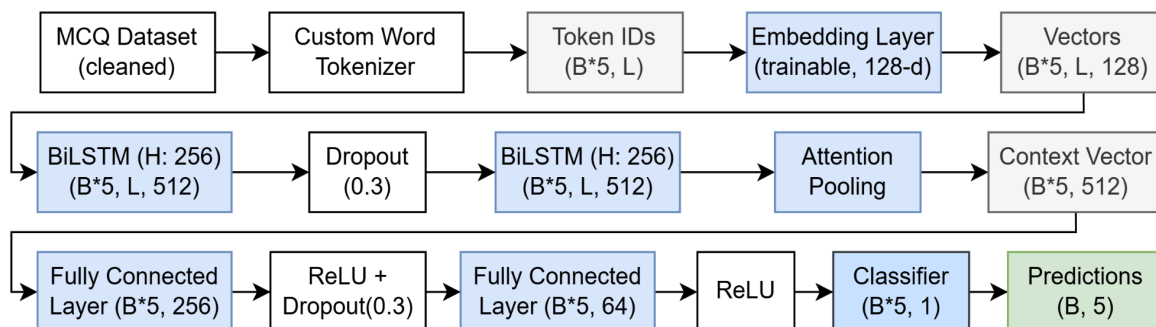
- Since it is a ML based model the training is very fast with very low computational cost.
- It set a strong baseline with 0.7502 MAP@3 score on public test data.

5.2 Model 2: Custom Deep Learning Model (BiLSTM)

A custom attention-based Bidirectional Long Short-Term Memory (BiLSTM) network was developed, which learns the word representations from the dataset using a custom vocabulary and trainable word embeddings. The model processes each question-option pair independently and predicts the relevance score for every candidate answer.

Architecture:

- **Custom Word Tokenizer:** A custom vocabulary is constructed from the dataset, each word is mapped to unique indices. The tokenizer converts every question-option pair into a sequence of token ids.
- **Embedding Layer:** The tokens were converted into trainable dense vectors, which represent the semantics.
- **Stacked Bidirectional LSTM Layers:** Two BiLSTM layers were used to capture the contextual information.



- **Attention -Based Pooling Layer:** Assigns importance weight to every token in sequence by focusing on the informative tokens, removing weights of padded tokens (via the attention_mask generated by WordTokenizer) and compresses the entire sequence into meaningful context vectors via weighted softmax sum.
- **Classification Head:** Context vector is passed through a multi-layer Fully Connected Neural Network with ReLU activation and dropout regularization to produce relevance scores (logits) for each question-option pair.

- **Predictions:** Since each question contains 5 candidate options, the predicted scores were reshaped to (B, 5). During inference the top 3 logits were selected as final prediction.

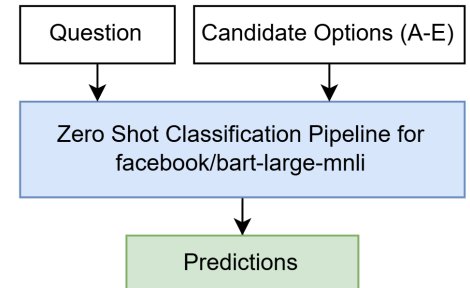
Salient Features:

- Effectively models the semantic relationships between questions and options with relatively low complexity.
- The embedding vectors were optimized during training, enabling the model to learn as per the task.
- Attention based pooling selectively gives more weights to informative words instead of treating every token equally, making a better sentence representation.
- Dropout layers reduce the chances of overfitting and improves the generalization.

5.3 Model 3: Zero-Shot BART

Architecture:

- To check the generalization capability of large pretrained models without task-specific training, a Zero-Shot pipeline from the Hugging Face was used to evaluate using the bart-large-mnli [5] model.
- For each MCQ the question was paired with each of the options to form individual pairs which were processed using the bart pipeline.
- BART is an encoder-decoder transformer sequence to sequence model.

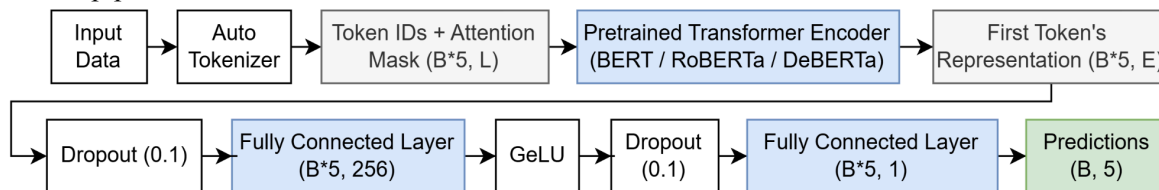


Salient Features:

- The model performs inference directly using pretrained knowledge without any training, eliminating the computational costs.
- BART uses both encoder and decoder, which can capture better contextual information.
- It was trained over natural language which makes it capable of understanding the questions.

5.4 Fine-Tuned Transformer Models

Three similar variants of transformer models (i.e BERT, RoBERTa, DeBERTa-v3) were fine-tuned with a common fine-tuning and classification pipeline.



Common architecture:

- For each question the question prompt was independently paired with options and processed.
- **Auto Tokenizer:** The inputs were tokenized via model specific tokenizers which were used at the time of pre-training and provided via the hugging face API. This provided token ids and attention masks.
- Token ids and attention masks were passed through the encoder to get the contextual representation from which the first token's representation is used, as it summarizes the context of all other tokens.
- A classification head is added on top of the encoder's output which consist of two fully connected layers with GeLU activation and dropout regularization, producing a single relevance score for each pair.

Salient Features:

- **Pretrained Language Knowledge:** Since these transformers were pretrained they already understand the representations, and further task specific fine-tuning makes them adapt better for the mcq answering task.
- **Self-Attention:** These transformer models use the self-attention mechanism [19] to learn the context by learning dependencies between all tokens in a subsequence, allowing the model to capture long contextual dependency.
- **Parallel Computation:** Transformer models process all tokens simultaneously, resulting in faster training.

5.4.1 Model 4: BERT

BERT (Bidirectional Encoder Representations from Transformer) [11] is an encoder-only transformer model. It used WordPiece tokenizer and was pretrained via raw text (unsupervised) using Masked Language Modeling (MLM) and Next Sentence Prediction (NSP) objectives. It learns the bidirectional (both left and right) contextual information.

5.4.2 Model 5: RoBERTa

RoBERTa (Robustly Optimized BERT Pretraining Approach) [2] uses the same architecture of BERT but it has improvements in the pre-training strategy. It removes the Next Sentence Prediction (NSP) objectives and only trains on Masked Language Modeling (MLM) with relatively larger datasets. It also used dynamic masking to change the masking tokens dynamically. It used a Byte-Pair Encoding tokenizer.

5.4.3 Model 6: DeBERTa-v3

DeBERTa-v3 (Decoding-enhanced BERT with Disentangled Attention) [3][4] extends the BERT architecture by introducing disentangled attention, which separates token content from position before computing attention scores. It uses SentencePiece

tokenizer.

Since DeBERTa-v3 is a relatively large model, we added weight decay (0.1) in the AdamW optimizer while fine-tuning.

5.5 Common Training Strategy:

To ensure fair comparison among all the modes, a common training framework was implemented using Pytorch Lightning [12] except a minor optimizer modification for DeBERTa-v3. The common training consists of:

- **Optimizer:** AdamW optimizer [9] was used for the parameter optimization.
- **Gradient Clipping:** A gradient clipping threshold of 1.0 was applied to the trainer to improve the training stability and prevent exploding gradients.
- **Early Stopping:** The training was eliminated if the validation MAP@3 score did not improve for five consecutive epochs.
- **Checkpointing:** The model which achieves the highest validation MAP@3 score was saved for inference.
- **Layer Freezing:** The pre-trained pooler layer of the Hugging Face transformer models was frozen during fine-tuning. The classification head operates on the first token's representation ([CLS] embedding) not the pooler output; the pooler layer was unused. Leaving it unfrozen was causing Distributed Data Parallel (DDP) issues (2 GPUs), as DDP requires every trainable parameter to receive a gradient during backpropagation.

6. Performance & Comparative Analysis

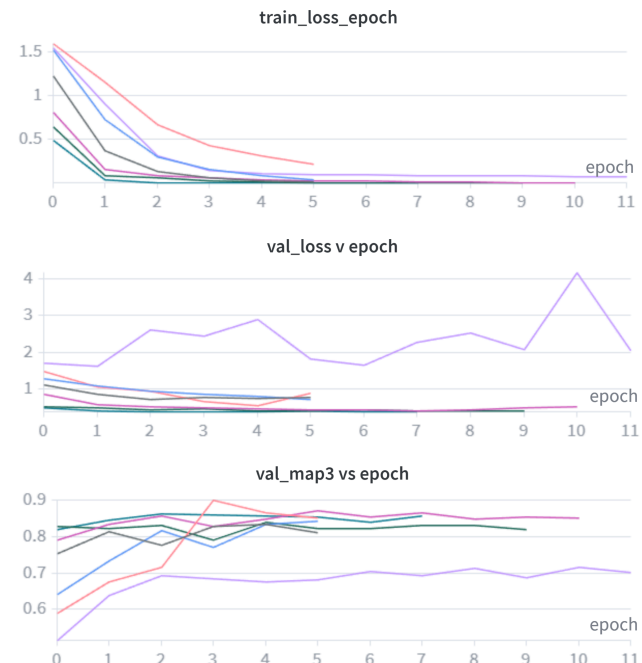
6.1 Evaluation Metrics

To evaluate the model performance following metrics were used throughout the experiments:

- **Validation Loss:** Cross-Entropy Loss is used to measure the optimization during training. Lower loss means better model performance.
- **Top-1 Accuracy:** Measures the percentage of samples for which the correct option is predicted at 1st index.
- **Top-3 Accuracy:** Measures the percentage of samples in which the correct option is there in top 3 predictions.
- **Mean Average Precision at 3 (MAP@3):** This is the main official evaluation metrics used in the kaggle competition, unlike accuracy this measures both the correctness as well as ranking of the predicted answers. Models will get a higher MAP@3 score when the correct answer is ranked closer to first position.
- **Epoch Duration & Training Time:** To analyse how much time each training epoch takes and how much time the whole training takes the epoch duration and training time is logged.

6.2 Training Performance

- All transformer models rapidly converge during initial epochs. The average training loss sharply decreases during the first two to three epochs and stabilizes thereafter.
- RAG based models showed convergence compared to standard transformer models.
- The convergence of custom BiLSTM shows consistent reduction in training loss but it is much slower than transformer models, since it is learning from scratch.
- The transformer models maintain relatively low val_loss. But the BiLSTM model shows higher validation loss due to its low ability to generalise, this is expected because its learning relies only on the training data.
- There is no significant overfitting observed in the transformer models as its val_loss remains low and stable.
- The pretrained transformers consistently outperform the BiLSTM, the DeBERTa-v3 archives the highest validation MAP@3, followed closely by BERT and RoBERTa.
- The RAG based models were slightly outperforming, showing the enhancement in results due to better context.



6.3 Comparative Report

Model Name	Training Time / Epoch	Validation Accuracy	Validation Accuracy@3	Validation MAP@3	Kaggle Score
DeBERTa v3 base + RAG	299.8659	0.8371	0.9365	0.8567	0.7502
RoBERTa + RAG	113.0151	0.8034	0.8918	0.8205	0.7465
BERT + RAG	112.0905	0.8538	0.9211	0.8529	0.7444
DeBERTa v3 base	85.3602	0.8507	0.9683	0.8514	0.7485
RoBERTa	45.8014	0.8118	0.9466	0.8433	0.7294
BERT	130.3287	0.7016	0.9506	0.8105	0.7394
BiLSTM01	6.3537	0.6398	0.8117	0.7023	0.6995
ZeroShot-Bart-Large-MNLI	-	-	-	-	0.4996
Classical-TFIDF-LightGBM	-	-	-	-	0.7502

The Classical ML Based LightGBM model outperformed many of the deep learning models, but in between of the deep learning models, the pretrained transformer based model on zero shot gives the lowest scores, whereas after fine-tuning the score of transformer based models were higher. The RAG pipeline provides only marginal improvement.

The ML Based LightGBM and the custom BiLSTM model trains the fastest as it has a lower number of parameters to train and the DeBERTa with the RAG pipeline takes the longest time to train.

The data shown above is from the leakage-free experiments, but initially there was data leakage in my experiments and the models which were trained with that issue have significantly higher kaggle scores than these ones i.e.

(Roberta + RAG - 0.75311, BERT - 0.75228, RoBERTa - 0.75187, DeBERTa - 0.7485)

6.4 Kaggle Performance

The highest public leaderboard score achieved during the competition was **0.75311**, resulting in the public leaderboard rank of **610**. The submitted model corresponds to RoBERTa + RAG..

7. Conclusion & Future Work

Key Learnings:

This project provided practical experience in designing, implementing and evaluating deep learning models for the NLP tasks, fine tuning transformer based models, and retrieval-augmented generation systems. Practical experience was gained to use various libraries and tools. Several models were implemented, and It was observed that increasing the model size alone after a certain point doesn't improve the performance, it is more dependent on the preprocessing and augmentation steps.

Challenges Faced:

One of the major challenges I faced was data leakage in the validation pipeline. Initially the experiments produced unrealistic validation scores but not performing on kaggle because duplicate and semantically similar questions appear in both training and validation data. This issue I addressed by introducing a semantic grouping strategy by using sentence embedding and StratifiedGroupKFold before splitting the dataset.

Areas of improvement:

The model's performance can be improved by adding a better knowledge base in the RAG pipeline, currently it is just from the training data itself, but it can be changed to something like wikipedia, etc. which provides better context. Even though we can connect it to its API, during inference for each question we can do something like, search the wikipedia page using the wikipedia API, then retrieve the most relevant doc from the page and use that as a context.

8. References

- [1] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," [arXiv:1810.04805](https://arxiv.org/abs/1810.04805), 2018.
- [2] Y. Liu et al., "RoBERTa: A Robustly Optimized BERT Pretraining Approach," [arXiv:1907.11692](https://arxiv.org/abs/1907.11692), 2019.
- [3] P. He, X. Liu, J. Gao, and W. Chen, "DeBERTa: Decoding-enhanced BERT with Disentangled Attention," [arXiv:2006.03654](https://arxiv.org/abs/2006.03654), 2020.
- [4] P. He, J. Gao, and W. Chen, "DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing," [arXiv:2111.09543](https://arxiv.org/abs/2111.09543), 2021.
- [5] M. Lewis et al., "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," [arXiv:1910.13461](https://arxiv.org/abs/1910.13461), 2019.
- [6] Y. Ke et al., "LightGBM: A Highly Efficient Gradient Boosting Decision Tree," in Advances in Neural Information Processing Systems (NeurIPS), 2017.
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of EMNLP-IJCNLP, 2019. Models used: sentence-transformers/all-MiniLM-L6-v2; cross-encoder/ms-marco-MiniLM-L6-v2.
- [8] M. Douze, A. Guzhva, C. Deng, J. Johnson, G. Szilvasy, P.-E. Mazaré, et al., "The Faiss Library," [arXiv:2401.08281](https://arxiv.org/abs/2401.08281), 2024.
- [9] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in Proceedings of ICLR, 2019. [arXiv:1711.05101](https://arxiv.org/abs/1711.05101).
- [10] T. Wolf et al., "Transformers: State-of-the-Art Natural Language Processing," in Proceedings of EMNLP: System Demonstrations, 2020. (Hugging Face Transformers)
- [11] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), 2019.
- [12] W. Falcon et al., "PyTorch Lightning," GitHub repository, 2019. <https://github.com/Lightning-AI/lightning>
- [13] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.
- [14] L. Biewald, "Experiment Tracking with Weights and Biases," 2020. <https://www.wandb.com>
- [15] [Kaggle] "Smart MCQ Solver Challenge" <https://www.kaggle.com/competitions/smart-mcq-solver-challenge>
- [16] M. Schuster and K. Nakajima, "Japanese and Korean Voice Search," in Proceedings of ICASSP, 2012. (WordPiece)
- [17] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in Proceedings of ACL, 2016. (Byte-Pair Encoding)
- [18] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in Proceedings of EMNLP: System Demonstrations, 2018.
- [19] A. Vaswani et al., "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017.